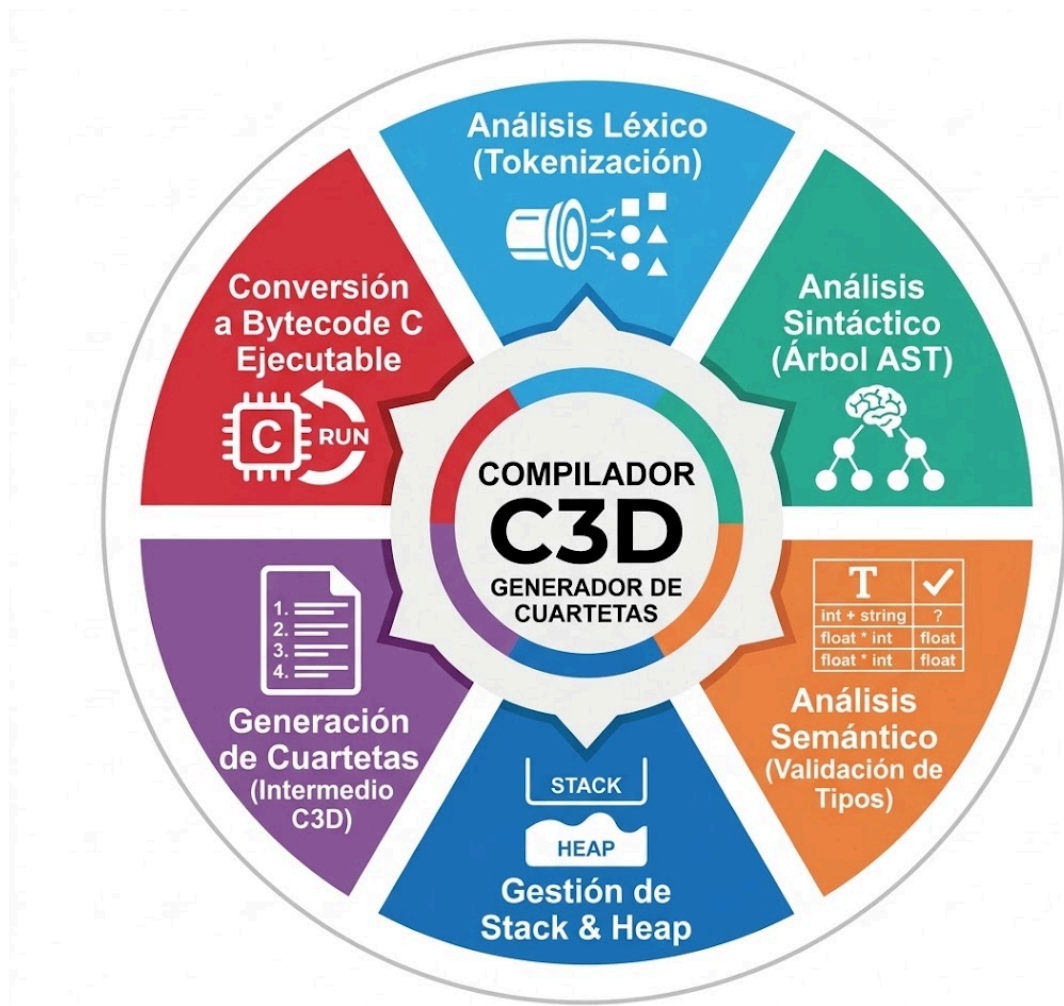


MANUAL DE USUARIO

IDE XS1



Javier Alejandro Mérida Gómez 202230638

26/09/2026

ÍNDICE

DESCRIPCIÓN GENERAL	3
CARACTERÍSTICAS DESTACADAS	3
EJECUCIÓN DEL SISTEMA	4
FUNCIONAMIENTO DE LA APLICACIÓN	4
1. ARCHIVOS	4
2. EDITOR DE TEXTO Y ÁRBOL DE TRABAJO	5
3. SECCIÓN DE ERRORES	7
4. BOTONES DE REPORTE	7
a. VER TABLAS DE SÍMBOLOS	8
b. VER TABLA DE TIPOS	8
c. VER TABLA DE COMPATIBILIDAD	8
d. VER ÁRBOLES AST	9
e. VER CUARTETAS	9
f. VER C3D	10
5. CÓDIGO C	11

DESCRIPCIÓN GENERAL

IDE capaz de soportar 3 lenguajes de alto nivel, capaz de poder validarlo léxica, sintáctica y semánticamente, para luego descompenerlo en código tres direcciones o código intermedio por medio de cuartetas y generar un archivo en lenguaje C, completamente funcional .

Mostrando las tablas, árboles abstractos, cuartetas, código tres direcciones y código , obtenidas durante el análisis.

CARACTERÍSTICAS DESTACADAS

- Análisis léxico, sintáctico y semántico de tres lenguajes completos.
- Coloreado automático de los lenguajes.
- IDE con interfaz amigable al usuario.
- Tabla de símbolos, de tipos y de computabilidad,
- Generación de código intermedio (Código 3D).
- Generación de archivo “C” funcional.
- Manejo completo de archivos (crear, cargar, guardar y exportar).

EJECUCIÓN DEL SISTEMA

Una vez descargada la aplicación, para iniciarla es necesario ejecutar el archivo .jar desde la línea de comandos, sigue estos pasos:

1. Abre una terminal en la ubicación donde se encuentra el archivo .jar.
2. Ejecuta el siguiente comando:

```
java -jar ProyectoUnoCompi2-1.0-SNAPSHOT.jar
```

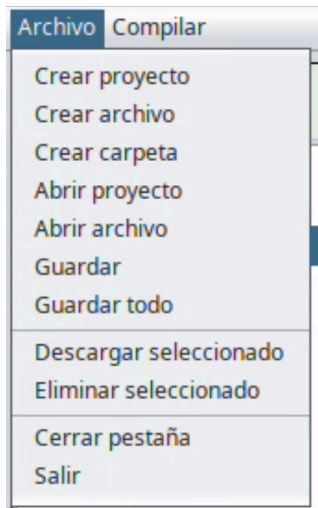
3. La aplicación se abrirá y quedará lista para usarse.

FUNCIONAMIENTO DE LA APLICACIÓN

1. ARCHIVOS

Se cuenta con un amplio menú para el manejo de archivos, con las siguientes opciones:

- Crear proyecto
- Crear archivo
- Crear carpeta
- Abrir proyecto
- Abrir archivo
- Guardar
- Guardar todo
- Descargar seleccionado
- Eliminar seleccionado
- Cerrar pestaña
- Salir



2. EDITOR DE TEXTO Y ÁRBOL DE TRABAJO

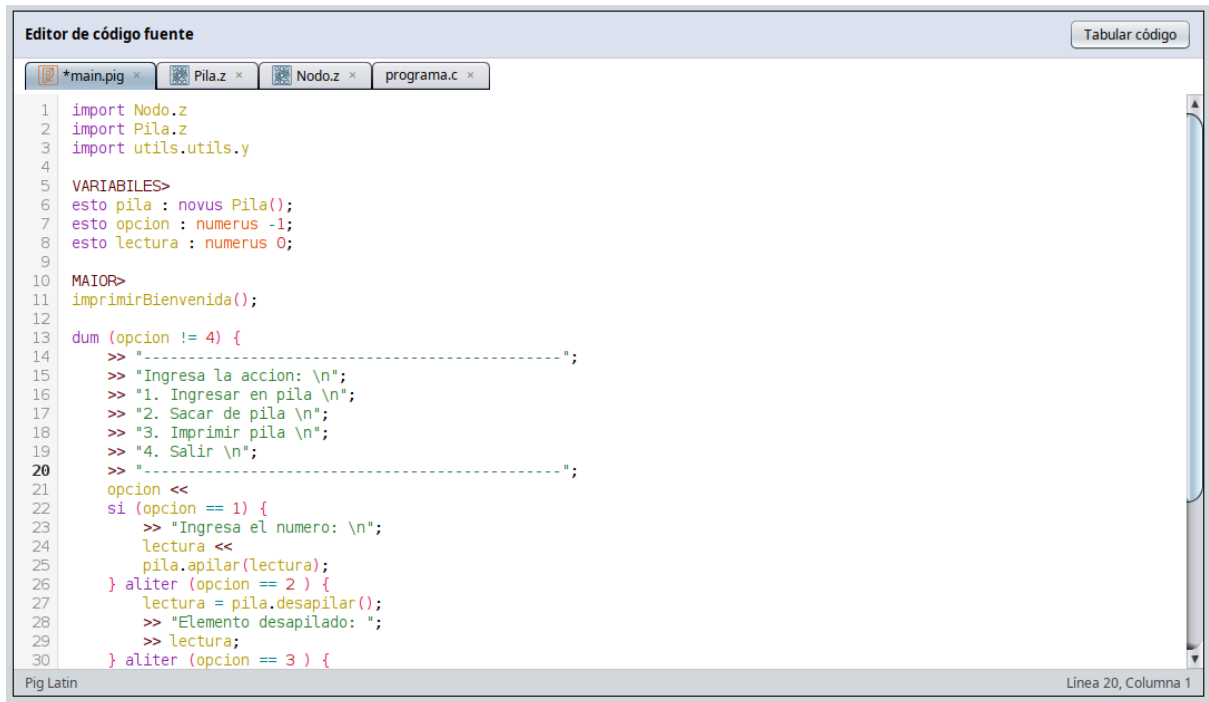
Para el editor de texto y árbol de trabajo se divide en dos secciones diferentes, que se detallan a continuación.

- Editor de texto
 - El editor de texto soporta varias pestañas de diferentes tipos de archivos.
 - Este cuenta con un coloreado de código, que se define en la siguiente tabla de referencia.

Categoría	Color
Palabras reservadas	Violeta #8E24AA
Tipos de dato	Naranja #E65100
Operadores	Cian #00838F
Identificadores	Amarillo #B39B00
Números	Rojo #C62828
Cadenas y caracteres	Verde #2E7D32
Agrupación	Rosa #E91E63
Secciones y funciones especiales	Vino #590202
Comentarios	Gris #8A8A8A (sin cursiva)
Todo lo demás	Negro #000000

- Cuenta con conteo de líneas y columnas para una mejor accesibilidad y uso del editor de texto.

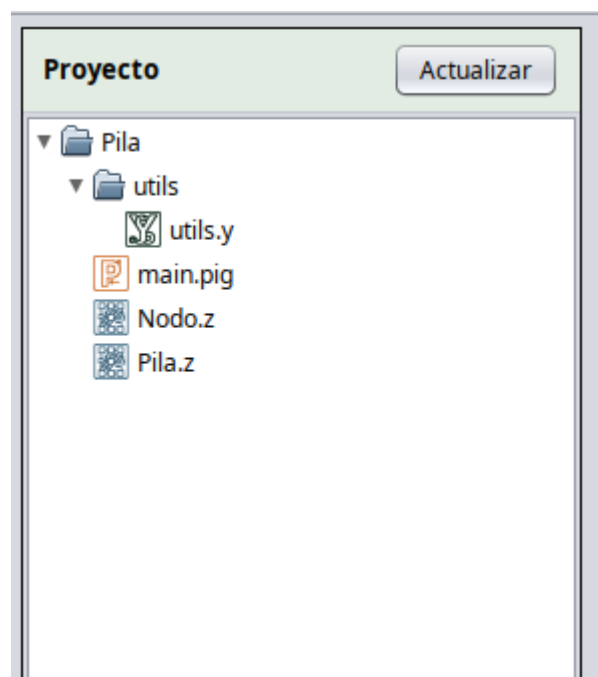
- Cuenta con el botón de “Tabular código”, esto es para tabular de forma automática el código de la pestaña que esté abierta de manera automática.



```
1 import Nodo.z
2 import Pila.z
3 import utils.utils.y
4
5 VARIABLES>
6 esto pila : novus Pila();
7 esto opcion : numerus -1;
8 esto lectura : numerus 0;
9
10 MAIOR>
11 imprimirBienvenida();
12
13 dum (opcion != 4) {
14     >> "-----";
15     >> "Ingresa la accion: \n";
16     >> "1. Ingresar en pila \n";
17     >> "2. Sacar de pila \n";
18     >> "3. Imprimir pila \n";
19     >> "4. Salir \n";
20     >> "-----";
21     opcion <<
22     si (opcion == 1) {
23         >> "Ingresa el numero: \n";
24         lectura <<
25         pila.apilar(lectura);
26     } aliter (opcion == 2) {
27         lectura = pila.desapilar();
28         >> "Elemento desapilado: ";
29         >> lectura;
30     } aliter (opcion == 3) {
```

Pig Latin Linea 20, Columna 1

- Árbol de trabajo
 - Dentro de esta sección vamos a visualizar la raíz de nuestro proyecto y ver las carpetas o archivos que tenemos.
 - Por cada extensión de archivo contamos con un icono que los identifica.



3. SECCIÓN DE ERRORES

Dentro de esta sección se enlistan todos los errores de tipo lexico, sintactico y semantico que se encuentran dentro del proyecto, para cada error se muestra lo siguiente:

- Tipo
- Archivo
- Línea
- Columna
- Lexema
- Descripción

Errores léxicos, sintácticos y semánticos						13 errores
Tipo	Archivo	Línea	Columna	Lexema	Descripción	
Semántico	main.pig	4	21		No se puede asignar un valor de tipo 'cadena' a uno de tipo 'entero'	
Léxico	main.pig	5	28	@	Símbolo no reconocido por el lenguaje: '@'	
Semántico	main.pig	6	27		No se puede asignar un valor de tipo 'cadena' a uno de tipo 'entero'	
Semántico	main.pig	9	8	+	El operador '+' no se puede aplicar a 'entero' y 'booleano'	
Semántico	main.pig	10	1	total	La variable 'total' no está declarada (o se usa antes de declararla)	
Semántico	main.pig	11	5		La condición debe ser de tipo booleano (se encontró 'entero')	
Semántico	main.pig	14	1	perge	'perge' solo se puede usar dentro de un ciclo	
Semántico	main.pig	15	4	saludar	La función 'saludar' no está definida	
Léxico	main.pig	15	20	#	Símbolo no reconocido por el lenguaje: '#'	
Léxico	utils.y	3	22	\$	Símbolo no reconocido por el lenguaje: '\$'	
Sintáctico	utils.y	6	34	salto de línea Falta ':' antes de salto de línea		
Sintáctico	utils.y	10	15	'entonces' Falta ')' antes de 'entonces'		
Léxico	utils.y	15	14	"sin cerrar" Cadena sin cerrar: falta la comilla doble (") de cierre en la misma línea		

4. BOTONES DE REPORTE

Dentro de esta sección vamos a encontrar todos los elementos generados dentro de nuestro análisis.

En ellos encontraremos:

- Tabla de símbolos
- Tabla de tipos
- Tabla de contabilidad
- AST
- Cuartetas
- Código tres direcciones

Ver tabla de símbolos
Ver tabla de tipos
Ver tabla de contabilidad
Ver árboles AST
Ver cuartetas
Ver C3D

a. VER TABLAS DE SÍMBOLOS

Aquí se verá la tabla de símbolos creada.

Tabla de símbolos								
Tabla de símbolos								
Nombre	Categoría	Tipo	Parámetros	Ámbito	Archivo	Línea	Desplazamiento	Tamaño
Cola	CONSTRUCTOR	Cola	()	Cola	Cola.z	6	-	1
encolar	METODO	void	(entero)	Cola	Cola.z	13	-	3
desencolar	METODO	entero	()	Cola	Cola.z	25	-	2
estaVacía	METODO	booleano	()	Cola	Cola.z	39	-	1
getTamaño	METODO	entero	()	Cola	Cola.z	43	-	1
toString	METODO	cadena	()	Cola	Cola.z	47	-	3
Nodo	CONSTRUCTOR	Nodo	(entero)	Nodo	Nodo.z	5	-	2
getDato	METODO	entero	()	Nodo	Nodo.z	10	-	1
getSiguiente	METODO	Nodo	()	Nodo	Nodo.z	14	-	1
setSiguiente	METODO	void	(Nodo)	Nodo	Nodo.z	18	-	2
mostrarMenu	FUNCION	void	()	global	utils/menu.y	2	-	0
opcionValida	FUNCION	booleano	(entero)	global	utils/menu.y	11	-	1
cola	VARIABLE	Cola	-	global	main.pig	6	0	1
opcion	VARIABLE	entero	-	global	main.pig	7	1	1
dato	VARIABLE	entero	-	global	main.pig	8	2	1
this	PARAMETRO	Nodo	-	Nodo.Nodo	Nodo.z	5	0	1
dato	PARAMETRO	entero	-	Nodo.Nodo	Nodo.z	5	1	1
this	PARAMETRO	Nodo	-	Nodo.getDato	Nodo.z	10	0	1
this	PARAMETRO	Nodo	-	Nodo.getSiguiente	Nodo.z	14	0	1

b. VER TABLA DE TIPOS

Aquí se verá la tabla de tipos creada.

Tabla de tipos						
Tabla de tipos						
Tipo	Categoría	Tamaño	Archivo	Atributo	Tipo del atributo	Desplazamiento
entero	PRIMITIVO	1	-	-	-	-
decimal	PRIMITIVO	1	-	-	-	-
cadena	PRIMITIVO	1	-	-	-	-
caracter	PRIMITIVO	1	-	-	-	-
booleano	PRIMITIVO	1	-	-	-	-
void	PRIMITIVO	1	-	-	-	-
Nodo	CLASE	2	Nodo.z	dato	entero	0
Nodo	CLASE	2	Nodo.z	siguiente	Nodo	1
Cola	CLASE	3	Cola.z	frente	Nodo	0
Cola	CLASE	3	Cola.z	fin	Nodo	1
Cola	CLASE	3	Cola.z	tamaño	entero	2

c. VER TABLA DE COMPATIBILIDAD

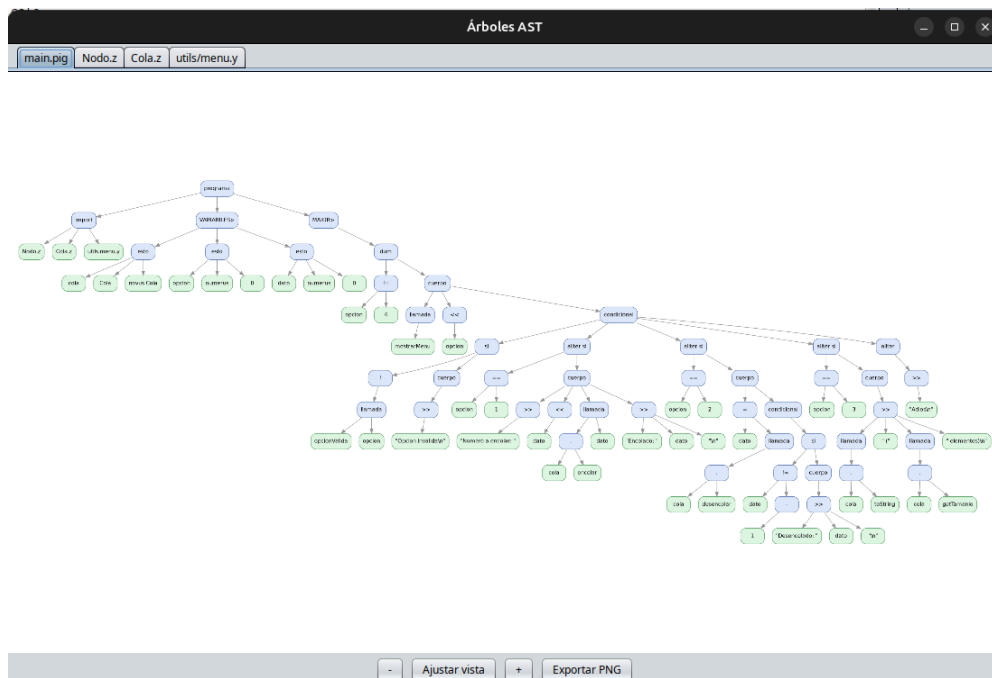
Aquí se verá la tabla de compatibilidad.

Izquierdo	Derecho	+	- * /	%	< > <= >=	== !=	&&	Asignar
numerus	numerus	numerus	numerus	numerus	bool	bool	error	sí
numerus	decimalis	decimalis	decimalis	error	bool	bool	error	error
numerus	textum	textum	error	error	error	error	error	error
numerus	littera	numerus	numerus	numerus	bool	bool	error	sí (implícita)
numerus	bool	error	error	error	error	error	error	error
decimalis	numerus	decimalis	decimalis	error	bool	bool	error	sí (implícita)
decimalis	decimalis	decimalis	decimalis	error	bool	bool	error	sí
decimalis	textum	error	error	error	error	error	error	error
decimalis	littera	decimalis	decimalis	error	bool	bool	error	sí (implícita)
decimalis	bool	error	error	error	error	error	error	error
textum	numerus	textum	error	error	error	error	error	error
textum	decimalis	textum	error	error	error	error	error	error
textum	textum	textum	error	error	error	bool	error	sí
textum	littera	textum	error	error	error	error	error	error
textum	bool	textum	error	error	error	error	error	error
littera	numerus	numerus	numerus	numerus	bool	bool	error	error
littera	decimalis	decimalis	decimalis	error	bool	bool	error	error
littera	textum	textum	error	error	error	error	error	error
littera	littera	numerus	numerus	numerus	bool	bool	error	sí
littera	bool	error	error	error	error	error	error	error
bool	numerus	error	error	error	error	error	error	error
bool	decimalis	error	error	error	error	error	error	error
bool	textum	error	error	error	error	error	error	error
bool	littera	error	error	error	error	error	error	error
bool	bool	error	error	error	error	bool	bool	sí

Un solo operando: -x: numerus → numerus, decimalis → decimalis, littera → numerus | !x / non x: bool → bool | ++ --: numerus → numerus, decimalis → decimalis, littera → littera

d. VER ÁRBOLES AST

Aquí se verán los árboles.



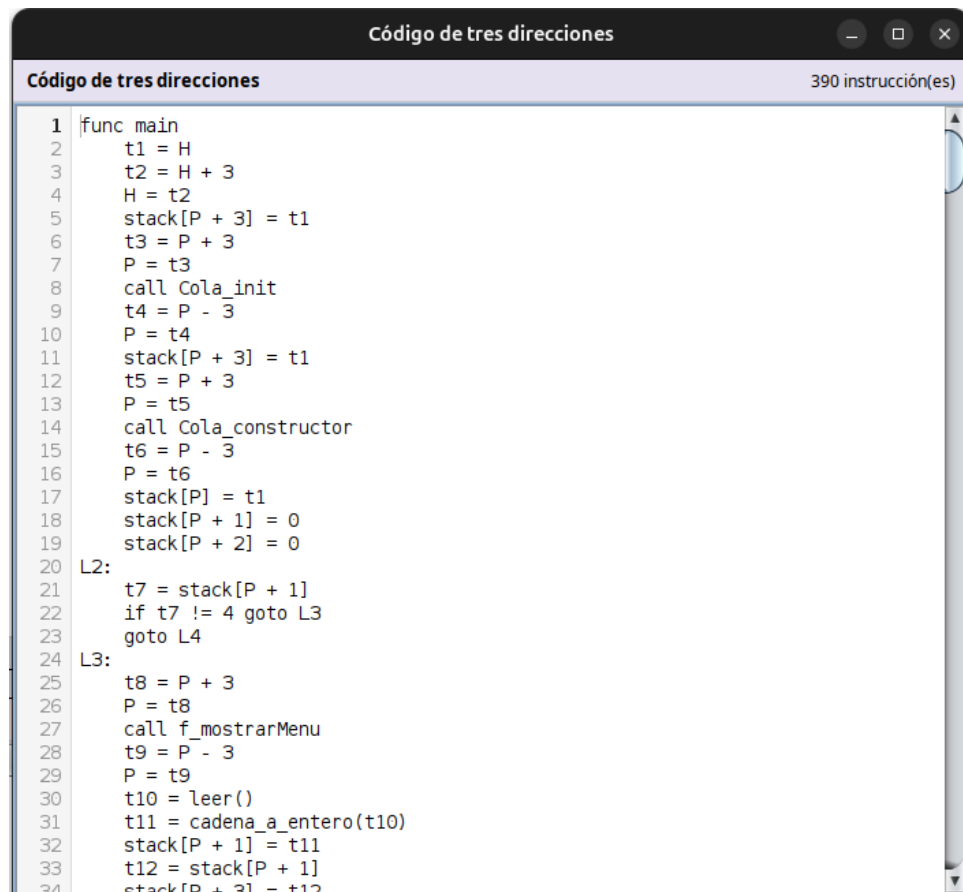
e. VER CUARTETAS

Aquí se verán las cuartetas.

Cuartetas					390 cuarteta(s)
#	Operador	Argumento 1	Argumento 2	Resultado	
1	func			main	
2	=	H		t1	
3	+	H	3	t2	
4	=	t2		H	
5	escribir_stack	t1	3	P	
6	+	P	3	t3	
7	=	t3		P	
8	call			Cola_init	
9	-	P	3	t4	
10	=	t4		P	
11	escribir_stack	t1	3	P	
12	+	P	3	t5	
13	=	t5		P	
14	call			Cola_constructor	
15	-	P	3	t6	
16	=	t6		P	
17	escribir_stack	t1	0	P	
18	escribir_stack	0	1	P	
19	escribir_stack	0	2	P	
20	label			L2	
21	leer_stack	P	1	t7	
22	if !=	t7	4	L3	
23	goto			L4	

f. VER C3D

Aquí se verá el código de tres direcciones .



```
1 func main
2   t1 = H
3   t2 = H + 3
4   H = t2
5   stack[P + 3] = t1
6   t3 = P + 3
7   P = t3
8   call Cola_init
9   t4 = P - 3
10  P = t4
11  stack[P + 3] = t1
12  t5 = P + 3
13  P = t5
14  call Cola_constructor
15  t6 = P - 3
16  P = t6
17  stack[P] = t1
18  stack[P + 1] = 0
19  stack[P + 2] = 0
20 L2:
21   t7 = stack[P + 1]
22   if t7 != 4 goto L3
23   goto L4
24 L3:
25   t8 = P + 3
26   P = t8
27   call f_mostrarMenu
28   t9 = P - 3
29   P = t9
30   t10 = leer()
31   t11 = cadena_a_entero(t10)
32   stack[P + 1] = t11
33   t12 = stack[P + 1]
34   stack[P + 3] = t12
```

5. CÓDIGO C

Después de la generación de todos los elementos separados se utilizan para la construcción de código c, con funcionalidad completa.

Cuando el código fue generado de manera automática, se puede copiar, compilar y descargar.

Código C generado

```
16 // Guarda un texto de C en el heap (un byte por celda y
17 double cadena_desde(const char *texto) {
18     int inicio = H;
19     for (; *texto; texto++) {
20         heap[H++] = (unsigned char) *texto;
21     }
22     heap[H++] = 0;
23     return inicio;
24 }
25
26 // Copia una cadena del heap a un texto de C
27 void a_texto(double cadena, char *texto, int maximo) {
28     int i = 0;
29     for (int c = (int) cadena; heap[c] != 0 && i < maximo; c++)
30         texto[i++] = (char) (int) heap[c];
31     }
32     texto[i] = 0;
33 }
34
35 // Un caracter (codigo Unicode) en UTF-8
36 void utf8(int c, char *texto) {
37     if (c < 0x80) {
38         texto[0] = c; texto[1] = 0;
39     } else if (c < 0x800) {
40         texto[0] = 0xC0 | (c >> 6); texto[1] = 0x80 | (c >> 0);
41     } else {
42         texto[0] = 0xE0 | (c >> 12); texto[1] = 0x80 | (c >> 6);
43         texto[2] = 0x80 | (c >> 0);
44     }
45 }
46
47 // Decimales como en Java: 8.5, 2.0
48 void formato_decimal(double d, char *texto) {
49     snprintf(texto, 64, "%.10g", d);
50     if (!strpbrk(texto, ".eni")) {
```

Copiar

Guardar .c

Compilar en C